Image Classification Model Using TensorFlow and Keras

Dataset: CIFAR-10 (10 image classes)
Model: Convolutional Neural Network (CNN)

Aim

Build an image classification model in four stages:

1. Load and preprocess image data.
2. Define the model architecture.
3. Train the model.
4. Estimate model performance.

Run all cells in order. The CIFAR-10 dataset downloads automatically the first time.

Stage A: Loading and preprocessing the image data

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical

print("TensorFlow version:", tf.__version__)

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]

print("Training images:", x_train.shape)
print("Testing images:", x_test.shape)

# Normalize pixel values from 0–255 to 0–1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Convert class labels to one-hot encoded vectors
y_train_cat = to_categorical(y_train, 10)
```

```
y_test_cat = to_categorical(y_test, 10)

# Display a sample of training images
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[int(y_train[i][0])])
    plt.axis("off")
plt.tight_layout()
plt.show()
```

```
TensorFlow version: 2.21.0
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 2716s 16us/step
Training images: (50000, 32, 32, 3)
Testing images: (10000, 32, 32, 3)
```



Stage B: Defining the model architecture

```
model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation="relu", padding="same"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
```

```
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax")
])

model.summary()

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 356,810 (1.36 MB)
Trainable params: 356,810 (1.36 MB)

Stage C: Training the model

```
history = model.fit(
    x_train,
    y_train_cat,
    epochs=10,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)
```

```
Epoch 1/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 115s 159ms/step - accuracy: 0.3717 - loss: 1.712
Epoch 2/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 112s 159ms/step - accuracy: 0.5272 - loss: 1.316
Epoch 3/10
```

```
704/704 ━━━━━━━━━━━━━━━━━━━━ 111s 158ms/step - accuracy: 0.5935 - loss: 1.146
Epoch 4/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 143s 160ms/step - accuracy: 0.6367 - loss: 1.035
Epoch 5/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 143s 162ms/step - accuracy: 0.6652 - loss: 0.954
Epoch 6/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 142s 162ms/step - accuracy: 0.6889 - loss: 0.888
Epoch 7/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 116s 164ms/step - accuracy: 0.7120 - loss: 0.831
Epoch 8/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 112s 159ms/step - accuracy: 0.7265 - loss: 0.781
Epoch 9/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 142s 159ms/step - accuracy: 0.7430 - loss: 0.731
Epoch 10/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 116s 165ms/step - accuracy: 0.7589 - loss: 0.685
```

Stage D: Estimating the model's performance

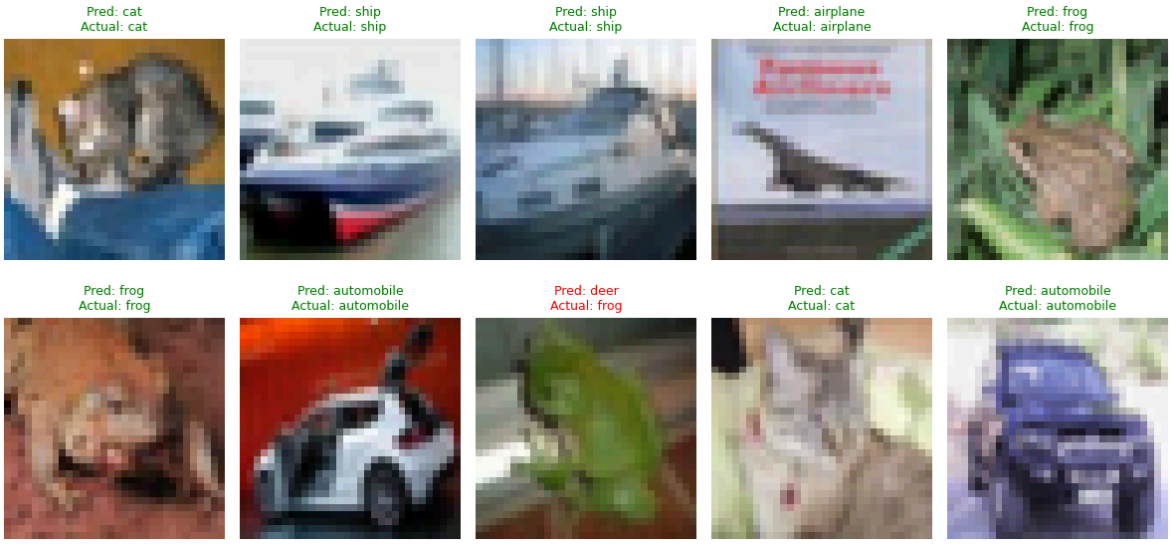
```
test_loss, test_accuracy = model.evaluate(x_test, y_test_cat, verbose=0)

print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

# Predict classes for a few test images
predictions = model.predict(x_test[:10], verbose=0)
predicted_labels = np.argmax(predictions, axis=1)
actual_labels = y_test[:10].flatten()

plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i])
    predicted_name = class_names[predicted_labels[i]]
    actual_name = class_names[actual_labels[i]]
    color = "green" if predicted_labels[i] == actual_labels[i] else "red"
    plt.title(f"Pred: {predicted_name}\nActual: {actual_name}", color=color,
    plt.axis("off")
plt.tight_layout()
plt.show()
```

```
Test loss: 0.8481
Test accuracy: 72.39%
```




```
# Plot training and validation accuracy/loss
epochs = range(1, len(history.history["loss"]) + 1)

plt.figure(figsize=(8, 5))
plt.plot(epochs, history.history["accuracy"], label="Training Accuracy")
plt.plot(epochs, history.history["val_accuracy"], label="Validation Accuracy
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(epochs, history.history["loss"], label="Training Loss")
plt.plot(epochs, history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

